CS 4400: Artificial Intelligence

First-order logic1

Propositional logic lacks expressivity

- Axioms of the wumpus world
  - $L^{t+1}_{[1,1]} \Leftrightarrow (L^{t}_{[1,1]} \wedge (\sim Forward^{t} \vee Bump^{t+1}))$
    $\vee (L^{t}_{[1,2]} \wedge (South^{t} \vee Forward^{t}))$
    $\vee (L^{t}_{[2,1]} \wedge (West^{t} \vee Forward^{t}))$
  - … repeat for all locations, all time steps, and all other fluents

- Can we generalize this in more compact ways?
  - "This holds for all variables"
  - Yes, with the first-order logic, and it's much more than just compact.

Problem of Propositional logic

☹ Propositional logic has very limited expressive power

- E.g., cannot say "pits cause breezes in adjacent squares“ except by writing one sentence for each square.
- We want to be able to say this in one single sentence:“for all squares and pits, pits cause breezes in adjacent squares.
- First order logic will provide this flexibility.

Contents

1. First order logic – objects and relations
2. Symbols and interpretation
3. Syntax and semantics
   - Sentences
   - Variables
   - Quantifiers
4. Examples
   - Kinship domain
   - Wumpus world

Formal Languages

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

1/4. First order logic

- Entities in FOL: objects and relations
  - More expressive than PL and is also closer to NL
- Objects: people, houses, numbers, Ronald McDonald, colors, baseball games
- Relations
  - Unary relation (=property): red, round, bogus, prime
  - N-ary: is a brother of, is bigger than, comes between
  - Functions: father of, best friend, beginning of
- Almost any assertion can be expressed with objects and relations

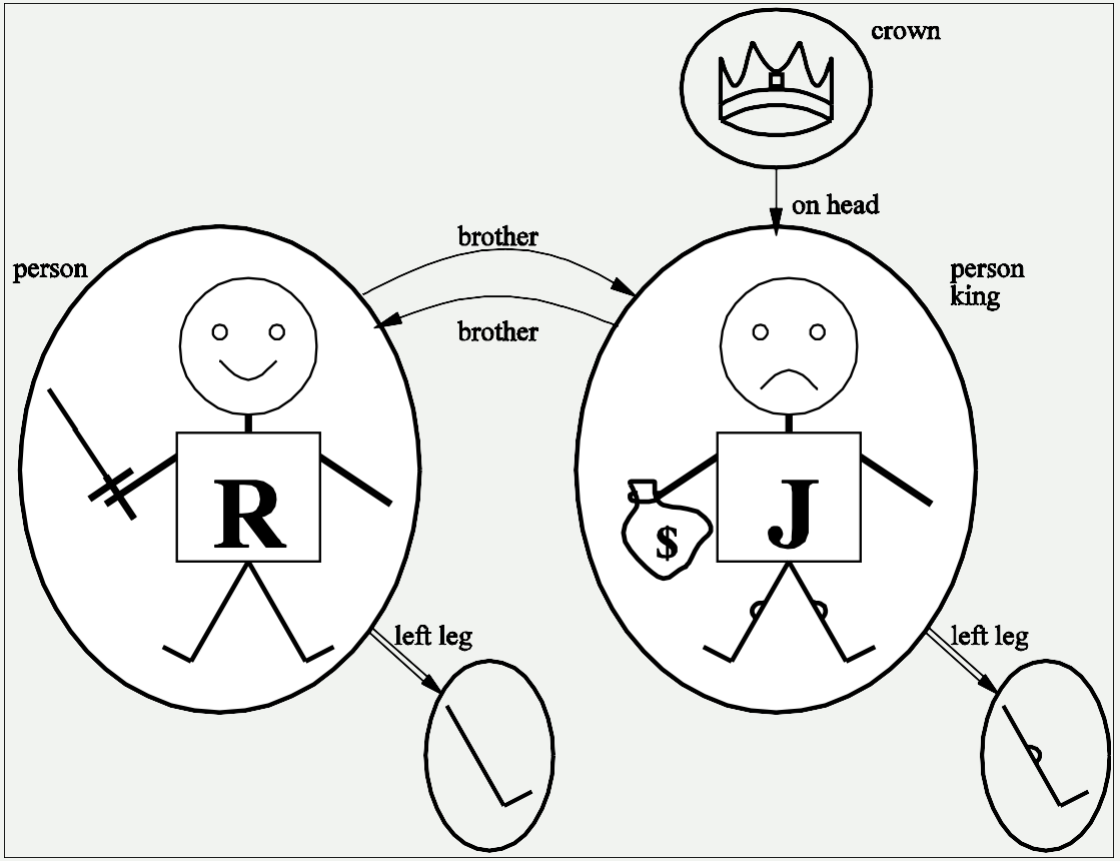
Examples

- “One plus two equals three.”
  - Objects: one, two, three
  - Binary relation: equals
  - Function: plus
- “Squares neighboring the wumpus are smelly.”
  - Objects: wumpus, squares
  - Unary relation: smelly
  - Binary relation: neighboring

A domain for first order logic

- 5 objects, 2 binary relations, 2 unary relations, 1 unary function

A model containing five objects, two binary relations, three unary relations (indicated by labels on the objects), and one unary function, left-leg.



- Objects:
  - Richard the Lionheart, King of England, 1189-1199
  - Richard's younger brother, the evil King John, 1199-1215
  - Left legs of Richard and John
  - A crown
- Unary relations: person, king
- Binary relations: brother, on-head
- Unary function: left leg

2/4. Symbols and interpretation

- Symbols represent entities (=objects and relations) and are basic syntactic elements
- Constant symbols: objects
  - *Richard*, *John*
- Predicate symbols: unary/binary/n-ary relations
  - *Brother(A,B)*, *OnHead(A,B)*, *Person(A)*, *King(A)*
- Function symbols
  - *LeftLegOf(A), SqrtOf(A)*

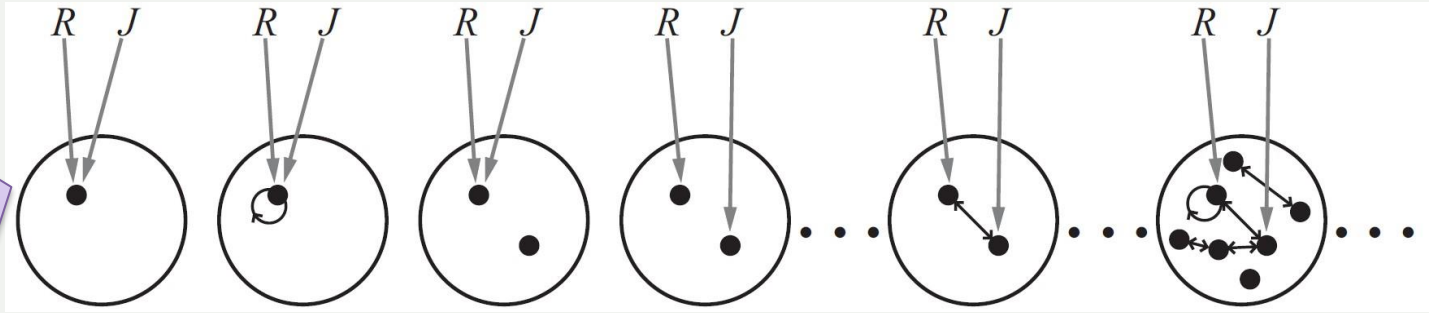
Interpretation

- Interpretation: mapping of symbols to objects and relations
- Suppose we have symbols *R*, *J*, *Bro*(,), *L*( )
  - In one interpretation,
    - *R* refers to “Richard the Lionheart”
    - *J* refers to “John the evil King”
    - *Bro(,)* refers to the “brotherhood relation”
    - *L( )* refers to the function “left leg of”
  - In another interpretation,
    - *R* refers to the “crown”, *J* to “King John’s left leg”?

Interpretation

- PL/FOL: a sentence is true/false for each model
  - PL: model = assignment of true/false to all symbols
  - FOL: model = assignment of objects & relation to all symbols (=interpretation)
    - Example: two constant symbols and a binary relation

The interpretation of each constant symbol is shown by a gray arrow. Within each model, the related objects are connected by arrows



  - In FOL, inference by enumeration is not feasible

3/4. FOL syntax / semantics

- Terms
- Atomic sentences
- Complex sentences
- Variables
- Universal quantifiers
- Existential quantifiers
- Equality

Terms

- Term is a logical expression to refer to objects

  Term = Constant symbol | *Variable* | *Function(term,…)*

- Examples
  - *KingJohn*
  - *x, y*
  - *LeftLegOf*
- Why use function? *it is not always convenient to have a distinct symbol to name every object*
  - We can use *LeftLeg(Richard)* instead of calling John's left leg.

we might use the expression "King John's left leg" rather than giving a name to his leg. This is what function symbols are for: instead of using a constant symbol, we use LeftLeg(John)

Atomic sentences

Now that we have both terms for referring to objects and predicate symbols for referring to relations, we can put them together to make atomic sentences that state facts

- Sentences in logic state facts that are true or false

  Sentence = AtomicSentence | ComplexSentence

  Atomic sentence = predicate | predicate(term,…)

  | term = term

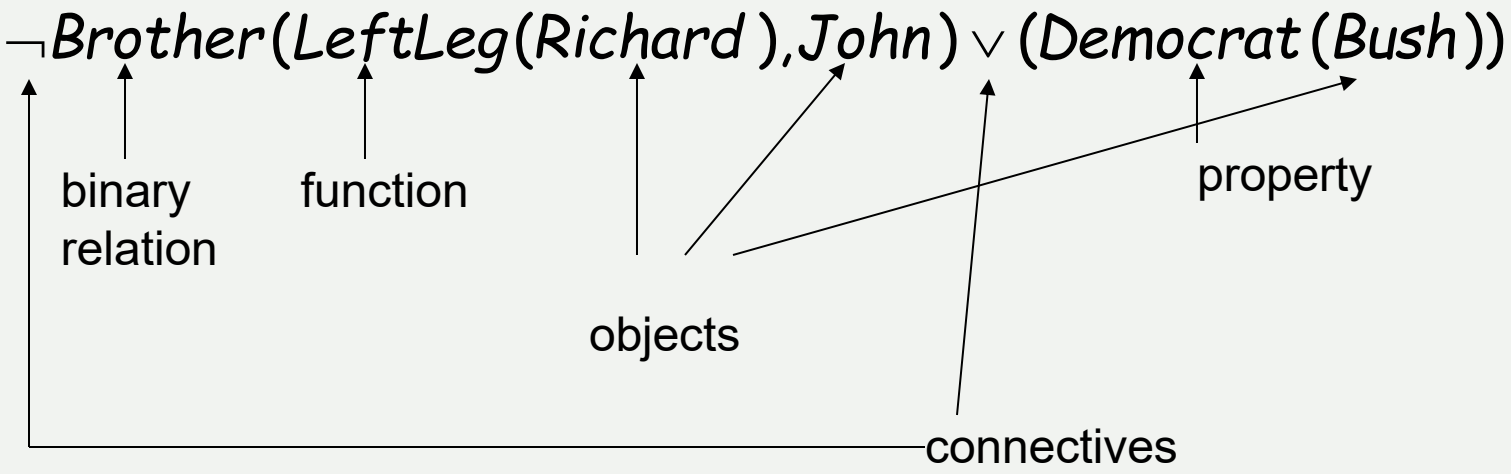
- Examples
  - *Brother(KingJohn, RichardTheLionheart)*
  - *Married(FatherOf(Richard), MotherOf(John))*
  - *>(Length(LeftLegOf(Richard)),Length(LeftLegOf(KingJohn)))*
- Semantics: an atomic sentence is true in a model, if the relations referred to by the predicate symbol holds among the objects referred to by the argument

Complex sentences

- *ComplexSentence* = (*Sentence*) | [*Sentence*]
  | ~*Sentence* : negation
  | (*Sentence* $\wedge$ *Sentence*) : conjunction
  | (*Sentence* $\vee$ *Sentence)* : disjunction
  | (*Sentence* $\Rightarrow$ *Sentence*) : implication
  | (*Sentence* $\Leftrightarrow$ *Sentence*) : biconditional
  | *Quantifier variable, … Sentence*
- Example
  - *~Brother(LeftLegOf(Richard), John)*
  - *King(Richard)* $\vee$ *King(John)*
  - *Sibling(KingJohn,Richard)* $\Rightarrow$ *Sibling(Richard,KingJohn)*

Complex sentences

- *We make complex sentences with connectives (just like in proposition logic).*



Variables & Quantifiers

- Variables appear as arguments to predicates
  - *Professor(y), King(x), BobbleHeadDoll(z)*
- Variables are meaningful only when used with quantifiers
  - A predicate is true for *all* instances of a variable
  - A predicate is true for *at least one* instance of a variable
  - A predicate is true for *no* instances of a variable

Quantifiers allow us to express properties of a collection of objects instead of enumerating them by name.

Universal quantifier

- ∀: reads "for all" or "for any"
- Meaning: true if all instantiations of variables make the enclosed sentence true
- Example
  - $\forall x\,(King(x) \Rightarrow Person(x))$: means "all kings are persons"
  - It is asserting the five sentences in Richard and John domain
    - Richard is a king ⇒Richard is a person
    - King John is a king ⇒King John is a person
    - Richard's left leg is a king ⇒Richard's left leg is a person
    - John's left leg is a king ⇒John's left leg is a person
    - The crown is a king ⇒The crown is a person
  - Since only the premise of 2nd sentence is true, it doesn't care about the rest

Universal quantifier

- Example
  - $\forall x\,(King(x) \Rightarrow Person(x))$: means "all kings are persons"
  - It is asserting the five sentences in Richard and John domain
    - Richard is a king ⇒Richard is a person
    - King John is a king ⇒King John is a person
    - Richard's left leg is a king ⇒Richard's left leg is a person
    - John's left leg is a king ⇒John's left leg is a person
    - The crown is a king ⇒The crown is a person

Let us look carefully at this set of assertions. Since, in our model, King John is the only king, the second sentence asserts that he is a person, as we would hope. But what about the other four sentences, which appear to make claims about legs and crowns? Is that part of the meaning of "All kings are persons"? In fact, the other four assertions are true in the model, but make no claim whatsoever about the personhood qualifications of legs, crowns, or indeed Richard. This is because none of these objects is a king. Looking at the truth table for ⇒ (Figure 7.8 on page 246), we see that the implication is true whenever its premise is false—*regardless* of the truth of the conclusion. Thus, by asserting the universally quantified sentence, which is equivalent to asserting a whole list of individual implications, we end up asserting the conclusion of the rule just for those objects for whom the premise is true and saying nothing at all about those individuals for whom the premise is false. Thus, the truth-table definition of ⇒ turns out to be perfect for writing general rules with universal quantifiers.

Existential quantifier

- ∃ : reads “there exists”, “for some”
- Meaning: there is at least one instantiation that satisfies the proposition
- Example
  - $\exists x\ (Crown(x) \wedge OnHead(x,John))$: means there’s (at least) one crown on John’s head
  - It is asserting that at least one of the five sentences is true
    - Richard is a crown ∧Richard is on John’s head
    - King John is a crown ∧King John is on John’s head
    - Richard’s left leg is a crown ∧Richard’s left leg is on John’s head
    - John’s left leg is a crown ∧John’s left leg is on John’s head
    - The crown is a crown ∧the crown is on John’s head
  - Note: $\exists x\ (Crown(x) \Rightarrow OnHead(x,John))$ is not very useful

Existential quantifier

- Example
  - $\exists x\ (Crown(x) \wedge OnHead(x,John))$: means there's (at least) one crown on John's head
  - It is asserting that at least one of the five sentences is true
    - Richard is a crown ∧Richard is on John's head
    - King John is a crown ∧King John is on John's head
    - Richard's left leg is a crown ∧Richard's left leg is on John's head
    - John's left leg is a crown ∧John's left leg is on John's head
    - The crown is a crown ∧the crown is on John's head
  - Note: $\exists x\ (Crown(x) \Rightarrow OnHead(x,John))$ is not very useful

The fifth assertion is true in the model, so the original existentially quantified sentence is true in the model. Notice that, by our definition, the sentence would also be true in a model in which King John was wearing two crowns. This is entirely consistent with the original sentence "King John has a crown on his head." [7]

Just as ⇒ appears to be the natural connective to use with ∀, ∧ is the natural connective to use with ∃. Using ∧ as the main connective with ∀ led to an overly strong statement in the example in the previous section; using ⇒ with ∃ usually leads to a very weak statement,

Common mistakes to avoid

$\Rightarrow$ *is the main connective with* $\forall$

$\wedge$ *is the main connective with* $\exists$

$\forall x, King(x) \Rightarrow Person(x) \quad x = \{Pete, Mary, tablespoon\}$

$\forall x, King(x) \wedge Person(x)$

$\exists x, King(x) \Rightarrow Person(x)$

$\exists x, King(x) \wedge Person(x)$

All of these must be true!
King(Pete) AND Person(Pete)
King(Mary) AND Person(Mary)
King(Tablespoon) AND Person(Tablespoon)

too strong

False!

One of these should be true!
if King(Pete) then Person(Pete)
if King(Mary) then Person(Mary)
If King(Tablespoon) then Person(Tablespoon) True!

too weak

Combining quantifiers

- Nested quantifiers
  - "Brothers are siblings": $\forall x \forall y\ Brother(x,y) \Rightarrow Sibling(x,y)$
  - $\forall x \forall y\ Brother(x,y) \Leftrightarrow Brother(y,x)$
  - Order matters (sometimes)
    - $\forall x \forall y$ is same as $\forall y \forall x$ (or $\forall yx$)
    - $\exists x \exists y$ is same as $\exists y \exists x$ (or $\exists yx$)
    - Q: $\forall x \exists y\ Loves(x,y)$ ≡ ? $\exists y \forall x Loves(x,y)$
- Negation rule (cf DeMorgan's)
  - $\forall x \sim Likes(x,Exams)$ is equivalent to $\sim \exists x Likes(x,Exams)$
  - $\forall x P(x) \equiv \sim \exists x \sim P(x)$, $\exists x P(x) \equiv \sim \forall x \sim P(x)$,
- All variables *must* be bound to a quantifier:

  $\exists x\ Dog(x) \vee Cat(y)$ is illegal

**As asserting that everyone dislikes exam is the same asserting there does not exist someone who likes them*

**Everyone agree P ≡ There is no one who does not agree P*

De Morgan's Law for Quantifiers

De Morgan's Rule

$$P \wedge Q \equiv \neg(\neg P \vee \neg Q)$$

$$P \vee Q \equiv \neg(\neg P \wedge \neg Q)$$

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

Generalized De Morgan's Rule

$$\forall x\, P \equiv \neg\exists x\, (\neg P)$$

$$\exists x\, P \equiv \neg\forall x\, (\neg P)$$

$$\neg\forall x\, P \equiv \exists x\, (\neg P)$$

$$\neg\exists x\, P \equiv \forall x\, (\neg P)$$

Rule is simple: if you bring a negation inside a disjunction or a conjunction, always switch between them (or →and, and → or).

De Morgan's Law for Quantifiers

$$\neg\forall x\, P(x) \quad \equiv \quad \exists x\, \neg P(x)$$

$$\neg\exists x\, P(x) \quad \equiv \quad \forall x\, \neg P(x)$$

Move negation inward, flipping the quantifiers:

$$\begin{aligned}
\neg\forall x\exists y\exists z\forall s\forall t\ \ P(x,y,z,s,t) \quad &\equiv \quad \exists x\neg\exists y\exists z\forall s\forall t\ \ P(x,y,z,s,t)\\
&\equiv \quad \exists x\forall y\neg\exists z\forall s\forall t\ \ P(x,y,z,s,t)\\
&\equiv \quad \exists x\forall y\forall z\neg\forall s\forall t\ \ P(x,y,z,s,t)\\
&\equiv \quad \exists x\forall y\forall z\exists s\neg\forall t\ \ P(x,y,z,s,t)\\
&\equiv \quad \exists x\forall y\forall z\exists s\exists t\quad \neg P(x,y,z,s,t)
\end{aligned}$$

Equality

**Equality symbol to signify that two terms refer to the same object*

- "Term = Term" is an atomic sentence
  - *FatherOf(John) = Henry*
  - Meaning: the object referred to by FatherOf(John) and the object referred to by Henry are the same
  - Examples
    - "Richard has at least two brothers"

      $\exists x,y\, Brother(x,Richard) \wedge Brother(y,Richard) \wedge \sim(x=y)$
    - "Richard has exactly two brothers"

      $\exists x,y\, [Brother(x,Richard) \wedge Brother(y,Richard) \wedge \sim(x=y)$
      $\wedge (\forall z\, Brother(z,Richard) \Rightarrow (z=x \vee z=y))]$

4/4. Kinship domain example

- Objects: people
- Relations
  - Unary: Male, Female
  - Binary: Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt, Uncle
  - Function: MotherOf, FatherOf
- Remember how are functions different from predicates?

Kinship domain axioms

- One's mother is one's female parent (in a traditional sense)

  $\forall m,c\ [\quad \Leftrightarrow \quad]$

- One's husband is one's male spouse (in a traditional sense)

  $\forall w,h\ [\quad \Leftrightarrow \quad]$

- Male and female are disjoint categories (in a traditional sense)

  $\forall x\ [\quad \Leftrightarrow \quad]$

- Parent and Child are inverse relations

  $\forall p,c\ [\quad \Leftrightarrow \quad]$

- A grandparent is a parent of one's parent

  $\forall g,c\ [\quad \Leftrightarrow \exists p \quad]$

- A sibling is another child of one's parent

  $\forall x,y\ [\quad \Leftrightarrow x \neq y \wedge \exists p \quad]$

Kinship domain axioms

- One's mother is one's female parent

  $\forall m,c \ (m = Mother(c) \Leftrightarrow Female(m) \wedge Parent(m,c))$

- One's husband is one's male spouse

  $\forall w,h \ (Husband(h,w) \Leftrightarrow Male(h) \wedge Spouse(h,w))$

- Male and female are disjoint categories

  $\forall x \ (Male(x) \Leftrightarrow \sim Female(x))$

- Parent and Child are inverse relations

  $\forall p,c \ (Parent(p,c) \Leftrightarrow Child(c,p))$

- A grandparent is a parent of one's parent

  $\forall g,c \ (Grandparent(g,c) \Leftrightarrow \exists p \ Parent(g,p) \wedge Parent(p,c))$

- A sibling is another child of one's parent

  $\forall x,y \ (Sibling(x,y) \Leftrightarrow x \neq y \wedge \exists p \ Parent(p,x) \wedge Parent(p,y))$

- Axioms
  - Basic factual information from which other conclusions can be derived
  - Plain facts: Male(Jim), Spouse(Jim,Laura)
  - Definitions: define other functions and predicates
- A core set of predicates
  - {Child, Spouse, Female}: vertical, horizontal, gender
  - All the other axioms can be defined from it
- Theorems are useful (but not necessary) sentences already entailed by the axioms
  - $\forall xy\ Sibling(x,y) \Leftrightarrow Sibling(y,x)$

Wumpus domain example

- Objects (symbols)
  - *Agent, Wumpus, Squares, Gold, … ,Stench, Breeze, Glitter, … , Left, Right,…*
- Relations (predicates)
  - Facts: *Stench(t), Breeze(t), Pit(r), HaveArrow(t), …*
  - Perceptions: *Percept([s,Breeze,g,m,c,t)*
  - Actions: *Turn(Left), Forward, Shoot,…*
  - *At(Agent or Wumpus, s, t)*
  - *Adjacent([a,b],[c,d])*

Wumpus world example

- Axioms (partial)
  - $\forall t,s,g,m,c\ Percept([s,Breeze,g,m,c],t) \Rightarrow Breeze(t)$
  - $\forall t,s,b,m,c\ Percept([s,b,Glitter,m,c],t) \Rightarrow Glitter(t)$
  - $\forall t\ HaveArrow(t+1) \Leftrightarrow (HaveArrow(t) \wedge \neg Action(Shoot, t))$
  - $\forall x,s_1,s_2,t\ At(x,s_1,t) \wedge At(x,s_2,t) \Rightarrow s_1=s_2$
  - $\forall a,b,c,d\ Adjacent([a,b],[c,d]) \Leftrightarrow (a=c \wedge (b=d-1 \vee b=d+1)) \vee (b=d \wedge (a=c-1 \vee a=c+1))$
  - $\forall s\ Breezy(s) \Leftrightarrow \exists r\ Adjacent(r,s) \wedge Pit(r)$
  - *Locations, orientation, …*
- Building a KB for a specific domain is called knowledge engineering

Knowledge engineering

- Knowledge-base in a specific domain has to be designed by knowledge engineering
  - Knowledge engineering represents information about the world in a form that can be utilized by a computer to solve complex tasks
- Steps of knowledge-engineering process
  - Identify the task - PEAS
  - Assemble relevant knowledge
  - Decide predicates, functions, constants
  - Encode general knowledge about the domain
  - Encode the specific problem instance
  - Test/debug with agents: ASK

Using FOL

- *We want to TELL things to the KB, e.g.*
  TELL(KB, $\forall x, King(x) \Rightarrow Person(x)$*)*

- *We also want to ASK things to the KB,*
  ASK(KB, $\exists x, Person(x)$*)*

- *The KB should return the list of x's for which Person(x) is true:* {x/John,x/Richard,...}

Examples

The kinship domain:

Brothers are siblings

$\forall x,y$ *Brother(x,y)* => *Sibling(x,y)*

One's mother is one's female parent

$\forall m,c$ *Mother(c)* = m $\Leftrightarrow$ *(Female(m)* $\wedge$ *Parent(m,c))*

"Sibling" is symmetric

$\forall x,y$ *Sibling(x,y)* $\Leftrightarrow$ *Sibling(y,x)*

Some may be considered axioms, others as theorems which can be derived from the axioms.

Translating English to FOL

Every gardener likes the sun.

```
(Ax) gardener(x) => likes(x,Sun)
```

You can fool some of the people all of the time.

```
(Ex) person(x) ^ ((At) time(t)) => can-fool(x,t))
```

You can fool all of the people some of the time.

```
(Ax) person(x) => ((Et) time(t) ^ can-fool(x,t))
```

All purple mushrooms are poisonous.

```
(Ax) (mushroom(x) ^ purple(x)) => poisonous(x)
```

No purple mushroom is poisonous.

```
~(Ex) purple(x) ^ mushroom(x) ^ poisonous(x)
```

or, equivalently,

```
(Ax) (mushroom(x) ^ purple(x)) => ~poisonous(x)
```

There are exactly two purple mushrooms.

```
(Ex)(Ey) mushroom(x) ^ purple(x) ^ mushroom(y) ^ purple(y) ^ ~(x=y) ^ (Az)
(mushroom(z) ^ purple(z)) => ((x=z) v (y=z))
```